```
    return node.updatePriority(node.getCounter())
                    #4
  elsif getSize() >= this.maxSize
                                    #5
    evictOneEntry()
                                                #6
  newNode ← this.elements.add(key, value, 1)
                            #7
         hashTable.set(key,
newNode)                                                  #8
  return true
```

Let’s take another look here at that code, copied over to listing 7.8 for your convenience, with some minimal simplifications:

Let’s imagine we have two concurrent calls to add two new entries (neither of them is in the cache yet) and our cache already contains `maxSize-1` elements: in other words, the next element will fill the cache to its maximum capacity.

Now, those two requests will proceed in parallel, and suppose call A is the first one to get to line 5 in listing 7.8: the condition is `false`, then it moves to line 7. But before call executes line 7, call B get to line 5, and – guess what? – the if condition is still `false`, so call B as well will move on to line 7, and execute it and line 8.

As a result, the cache will hold one element more than allowed: figure 7.12 shows this situation. The good news is that on line 5, we already do things the right way, and check if current size is greater or equal the max allowed size. If we were just checking for equality, this race condition would